

zip() and Slicing — Consecutive Pairs Pattern

What `zip()` does (plain English)

`zip()` takes two lists and walks through them **at the same time**, grabbing one item from each list per step, and pairs them up.

```
names = ["Ali", "Sara", "Bilal"]
ages  = [20, 22, 19]

for n, a in zip(names, ages):
    print(n, a)
```

Output:

```
Ali 20
Sara 22
Bilal 19
```

No `range(len(...))`, no manual indexing — `zip()` hands you one pair per loop, and stops automatically when the shorter list runs out.

The "consecutive pairs" trick

Goal: pair every element in a list with the element right after it (e.g. current check-in with the next check-in).

Step 1 — make a shifted copy using slicing

```
checkin_times[1:]
```

[1:] means "start from index 1 to the end" — so it's the same list, shifted one position to the left.

```
Original list:    540, 542, 561, 561, 598, 599
Sliced [1:]:      542, 561, 561, 598, 599
```

Step 2 — zip the original against the shifted copy

```
zip(checkin_times, checkin_times[1:])
```

Line the two lists up. Whatever ends up in the same row is automatically "current item" + "the item right after it":

```
List 1 (original):    540, 542, 561, 561, 598, ...
List 2 (shifted):      542, 561, 561, 598, ...
                        ↓      ↓      ↓      ↓      ↓
Pairs:                (540, 542) (542, 561) (561, 561) (561, 598)
```

`zip()` itself doesn't know these are "consecutive" — it just pairs column by column. **The slicing is what makes the columns line up as current + next.** `zip()` is just the mechanical pairing on top.

Full working example

Problem: flag any pair of consecutive check-in times with a gap < 5 min.

```
checkin_times = [540, 542, 561, 561, 598, 599, 640, 700, 700, 700]

for a, b in zip(checkin_times, checkin_times[1:]):
    gap = b - a
```

```
if gap < 5:
    print(f"Gap of {gap} min between check-in at {a}
```

Output:

```
Gap of 2 min between check-in at 540 and check-in at 542
Gap of 0 min between check-in at 561 and check-in at 561
Gap of 1 min between check-in at 598 and check-in at 599
Gap of 1 min between check-in at 700 and check-in at 701
Gap of 2 min between check-in at 701 and check-in at 703
```

Why this beats manual indexing

Manual indexing (<code>i, i+1</code>)	<code>zip()</code> + slicing
Easy to get off-by-one bugs	Handles the boundary automatically
<code>range(len(...)-1)</code> needed to avoid <code>IndexError</code>	<code>zip()</code> stops safely at the shorter list
Harder to read at a glance	Reads like plain English: "for a, b in pairs"

Key fact to remember: `zip()` always stops at the length of the *shorter* list. Since `checkin_times[1:]` is always one item shorter than `checkin_times`, you automatically get exactly `n - 1` pairs — no risk of running past the end of the list.